C++ Midway Project — Store Inventory Management System

A menu-driven inventory manager using classes, pointers, dynamic memory, a binary search and plain-text file storage.

Course: CIS-25 Programming Using C++ (L1-41736) · **Student:** H Cheng · **Repository folder:** `Midterm/`

---

1. What the program does

The program manages the product inventory of a small online store. It is the same application that has been built up across the course modules, brought together into one program.

When it starts it reads its records from `inventory.txt` and loads them into an array that it allocates on the heap. It then asks who is signing in. An **Admin** may add, delete and update records. A **Regular User** may only view and search them. From then on a menu of ten options runs in a loop until the user chooses Exit.

The records are always held sorted by product ID. That ordering is what makes the **binary search** on menu option 2 valid: it finds any record by halving the range it still has to consider, so 8 records need at most 4 comparisons rather than 8. The same search is reused to delete a record and to refuse a duplicate product ID. A second, linear search on option 3 matches descriptions as text, which gives a direct comparison between a search that can use sorted data and one that cannot.

Every prompt validates what is typed. Letters where a number belongs, a blank description, a negative quantity and a price of zero are all refused with an explanation, and the question is asked again. On exit the program writes the records back to `inventory.txt`, so the next run begins where this one ended.

2. How it is built

```
Midterm/
├── main.cpp          menu loop, switch/case, input validation, data-type screen
├── Product.h / .cpp   class 1: one inventory record
├── Inventory.h / .cpp   class 2: the dynamic array, binary search, file I/O
├── User.h / .cpp      class 3: User, with Admin and RegularUser
├── inventory.txt      the data file that is read and written
├── inventory_report.txt   the report the program writes
└── output.pdf          this report
```

> **Build and run**
> `clang++ -Wall -Wextra -std=c++17 main.cpp Product.cpp Inventory.cpp User.cpp -o main`
> `./main`
> Compiles with no warnings under both `clang++` and `g++`. Run from inside the `Midterm` folder so the program finds `inventory.txt`.

3. Required concepts and where each one is used

Concept	How this program uses it	Where to find it
Data types and data sizes	`bool`, `char`, `short`, `int`, `long`, `long long`, `float`, `double` and `std::string` are all measured with `sizeof` on menu option 8, together with the range of `int` and the digit precision of `float` against `double`.	`main.cpp` → `showDataTypes()`
File types (.txt)	Two plain-text files. `inventory.txt` is the pipe-delimited data file the program reads at start-up and rewrites on exit. `inventory_report.txt` is a formatted report written for a person to read.	`inventory.txt`, `inventory_report.txt`
Use of pointers	The record array is reached only through the pointer `Product *items`. The binary search returns a pointer to the match or `nullptr`, the price update writes through that pointer, deletion recovers an index by subtracting two pointers, and menu option 9 walks the array with pointer arithmetic.	`Inventory.h`, `Inventory.cpp` → `pointerWalk()`
Arrays and dynamic memory	The records live in a heap array created with `new Product[capacity]`. When it fills up, `resize()` allocates a larger block, copies the records across and frees the old one. The destructor releases the array with `delete[]`, so nothing leaks.	`Inventory.cpp` → `constructor, resize(), destructor`
Binary search algorithm	`binarySearchById()` searches the sorted array by product ID, halving the range at every step and reporting how many comparisons it used. It also backs deletion and the duplicate-ID check. A second, linear search over descriptions is included for contrast.	`Inventory.cpp` → `binarySearchById()`
Use of strings	`std::string` holds stock codes, descriptions and usernames. The program splits file lines on the `|` character, trims whitespace from typed input, converts text to upper case for a case-insensitive match and uses `find()` to test for a substring.	`Product.cpp` → `fromFileLine()`, `Inventory.cpp` → `findByDescription()`
File I/O	`ifstream` reads the data file a line at a time at start-up, skipping blanks, comment lines and malformed records. `ofstream` writes the data file and the report. Every file is closed after use.	`Inventory.cpp` → `loadFromFile()`, `saveToFile()`, `writeReport()`
At least two classes	Three: `Product` (one record, all data private, with a constructor, validating setters and several member functions), `Inventory` (owns the dynamic array and the searching, sorting and file work) and `User`, from which `Admin` and `RegularUser` inherit to carry the permissions.	`Product.h`, `Inventory.h`, `User.h`

Also demonstrated, from earlier modules:

Menu, loop and switch/case	A menu of ten options inside a loop, routed by a `switch` statement, which keeps running until Exit is chosen.
Input validation	Every prompt rejects bad input, explains the problem and asks again, so no typo can corrupt the data.
Encapsulation	All data members are private. Nothing outside a class can write to them except through a setter that validates the value first.
Inheritance and virtual functions	`Admin` and `RegularUser` inherit from `User`. The menu asks any user object what it may do and the virtual function answers for its real role.
Separate compilation	Four translation units with their own headers and include guards, linked into one program.

4. Screenshots of the program running

Each image is a capture of a real session. The typed input is visible after each prompt.

Screen 1 — Start-up, file load, role sign-in and the full product table

The program opens `inventory.txt`, reads 8 records into the dynamic array, asks which role to sign in as, then prints the whole inventory sorted by product ID. The footer line reports the array capacity, which is larger than the record count because the array doubles itself as it grows.

Midterm — ./main · File load, role sign-in, main menu, full product table (part 1 of 2)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

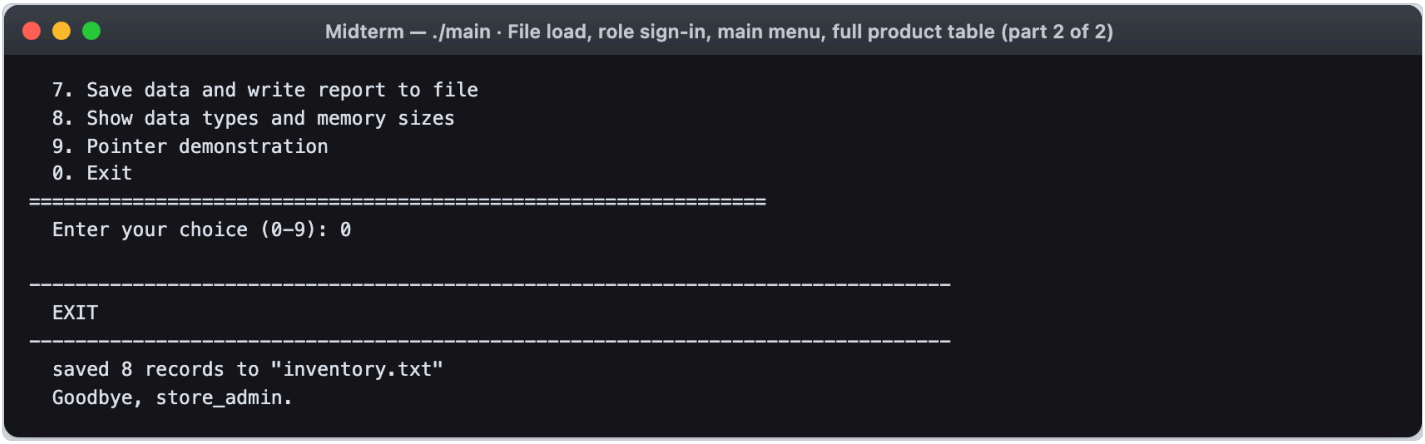
============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 1

--------------------------------------------------------------------------------
  ALL PRODUCTS (array kept sorted by product ID)
--------------------------------------------------------------------------------
    ------------------------------------------------------------------------------
    ID     STOCK     DESCRIPTION                              QTY   UNIT ($)   VALUE ($)
    ------------------------------------------------------------------------------
    105    71053     WHITE METAL LANTERN                        6       3.39       20.34
    118    84029G    KNITTED UNION FLAG HOT WATER BOTTLE        6       3.39       20.34
    150    22752     SET 7 BABUSHKA NESTING BOXES               2       7.65       15.30
    201    85123A    WHITE HANGING HEART T-LIGHT HOLDER         6       2.55       15.30
    233    22633     HAND WARMER UNION JACK                     6       1.85       11.10
    276    84029E    RED WOOLLY HOTTIE WHITE HEART.             6       3.39       20.34
    342    84406B    CREAM CUPID HEARTS COAT HANGER             8       2.75       22.00
    389    84879     ASSORTED COLOUR BIRD ORNAMENT             32       1.69       54.08
    ------------------------------------------------------------------------------
    8 records, sorted by product ID (array capacity 8)

  Total units: 72    Total value: $178.80

============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
```

Screen 1 — Start-up, file load, role sign-in and the full product table — continued (part 2 of 2)



Screen 2 — Binary search by product ID, one hit and one miss

Searching for ID 276 takes 2 comparisons and searching for the missing ID 999 takes 4, against 8 records. Each step prints its left, right and mid positions so the halving is visible. The program also states how many comparisons a linear search would have needed.

Midterm — ./main · Binary search by product ID (part 1 of 2)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
==============================================================
  Enter your choice (0-9): 2

--------------------------------------------------------------------------------
  BINARY SEARCH BY PRODUCT ID
--------------------------------------------------------------------------------
  Product ID to find: 276

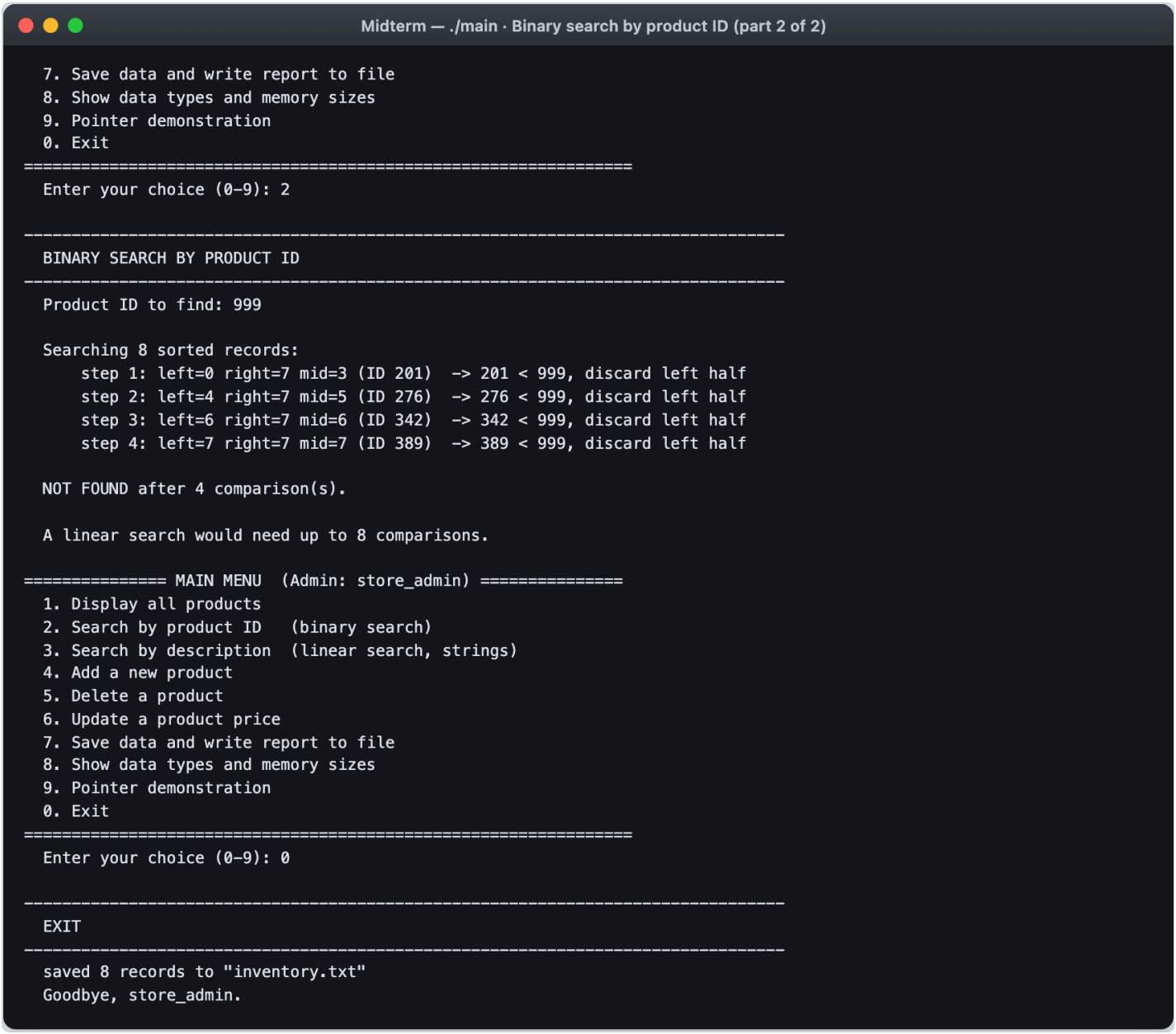
  Searching 8 sorted records:
      step 1: left=0 right=7 mid=3 (ID 201)  -> 201 < 276, discard left half
      step 2: left=4 right=7 mid=5 (ID 276)  -> MATCH

  FOUND after 2 comparison(s):
    Product ID:   276
    Stock Code:   84029E
    Description:  RED WOOLLY HOTTIE WHITE HEART.
    Quantity:     6
    Unit Price:   $3.39
    Stock Value:  $20.34

  A linear search would need up to 8 comparisons.

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
```

Screen 2 — Binary search by product ID, one hit and one miss — continued (part 2 of 2)



Screen 3 — Input validation, then a successful add

The add screen refuses the word `abc` where a number belongs, refuses a blank description, refuses a negative quantity and refuses a price of zero. Each refusal explains itself and asks again. Once every field is valid, the record is inserted at the position that keeps the array sorted.

Midterm — ./main · Input validation, then a successful add (part 1 of 2)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

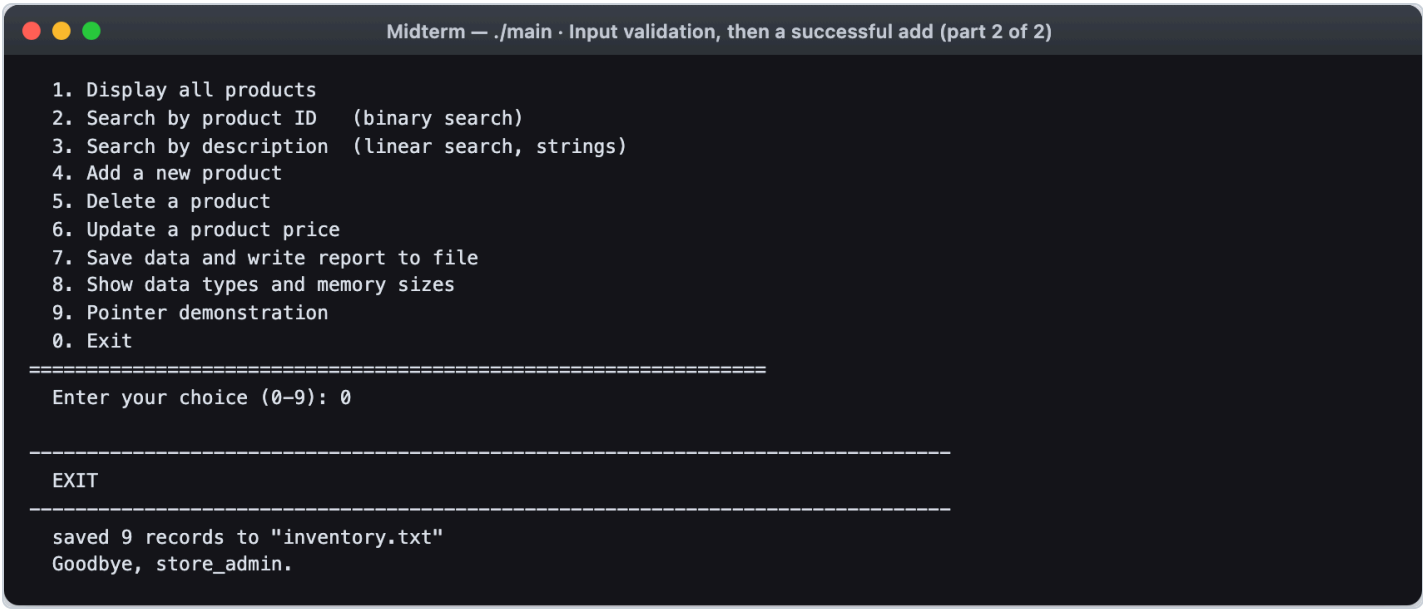
=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 4

--------------------------------------------------------------------------------
  ADD A NEW PRODUCT
--------------------------------------------------------------------------------
  Product ID (1-999999): abc
    [Invalid] That is not a whole number. Try again.
  Product ID (1-999999): 450
  Stock code:       TEST9
  Description:
    [Invalid] This cannot be left blank.
  Description:      GLASS STAR FROSTED T-LIGHT HOLDER
  Quantity (0-100000): -2
    [Invalid] Enter a number from 0 to 100000.
  Quantity (0-100000): 6
  Unit price:       $0
    [Invalid] The price must be greater than 0.00.
  Unit price:       $4.25

  Added and placed at sorted position 8 of 9.
    Product ID:   450
    Stock Code:   TEST9
    Description:  GLASS STAR FROSTED T-LIGHT HOLDER
    Quantity:     6
    Unit Price:   $4.25
    Stock Value:  $25.50

=============== MAIN MENU  (Admin: store_admin) ===============
```

Screen 3 — Input validation, then a successful add — continued (part 2 of 2)



Screen 4 — String search by description

A second search works on text instead of numbers. Descriptions are not sorted, so this one is linear: it compares record by record and ignores capitalisation, so the lower-case word “babushka” matches the stored upper-case description. The second search shows the not-found path.

Midterm — ./main · String search by description (part 1 of 2)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

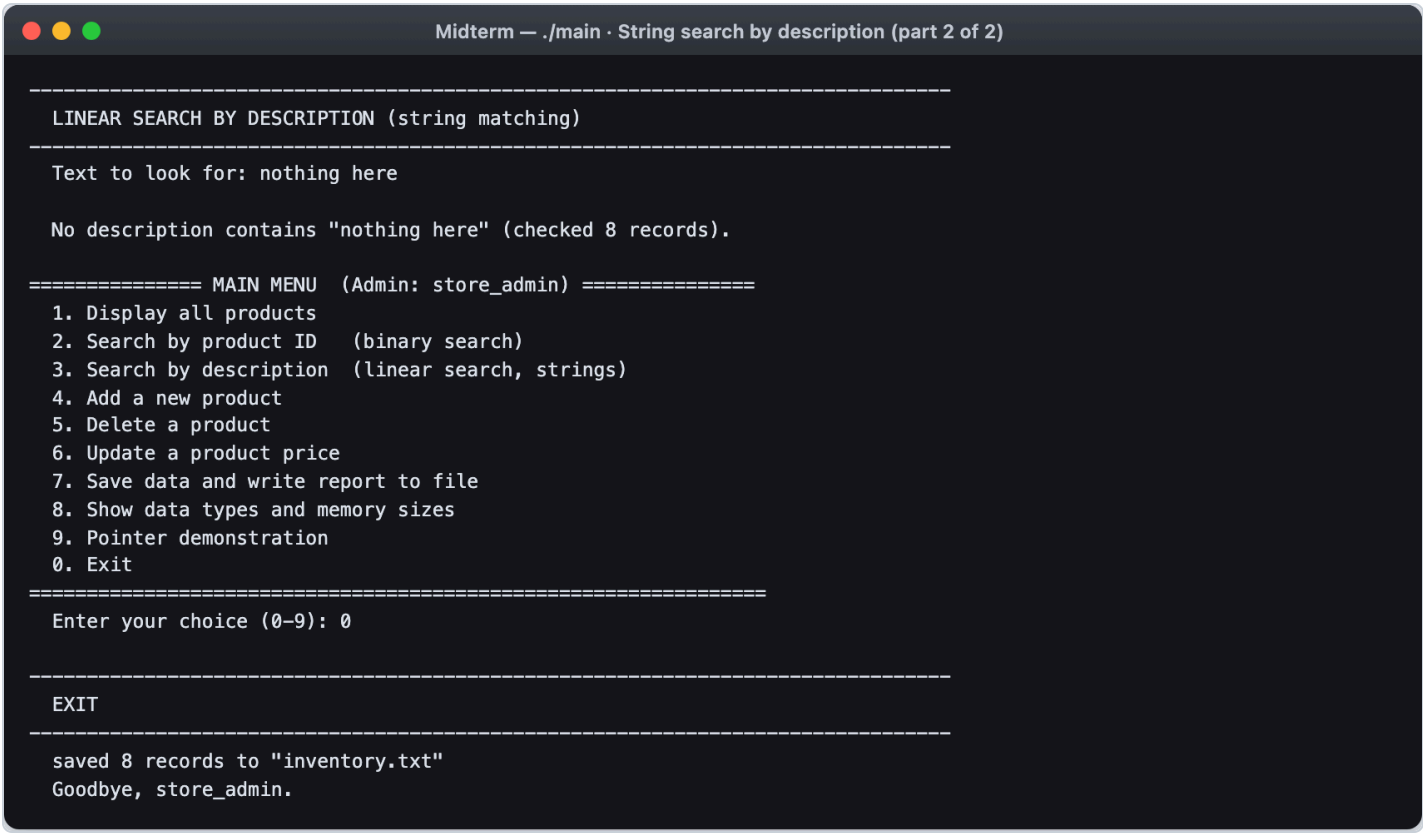
=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
==============================================================
  Enter your choice (0-9): 3

--------------------------------------------------------------------------------
  LINEAR SEARCH BY DESCRIPTION (string matching)
--------------------------------------------------------------------------------
  Text to look for: babushka

  FOUND after 3 comparison(s) (capitalisation ignored):
    Product ID:   150
    Stock Code:   22752
    Description:  SET 7 BABUSHKA NESTING BOXES
    Quantity:     2
    Unit Price:   $7.65
    Stock Value:  $15.30

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
==============================================================
  Enter your choice (0-9): 3
```

Screen 4 — String search by description — continued (part 2 of 2)



Screen 5 — Role restrictions

Signed in as a Regular User, menu options 4 and 5 are refused before any work is done, and the program names the role that would be required. Viewing and searching still work, so the restriction is on changing data, not on reading it.

Midterm — ./main · Regular User blocked from admin actions (part 1 of 2)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 2
  Enter your username: shopper_01

  Signed in as shopper_01 with the role: Regular User

============== MAIN MENU  (Regular User: shopper_01) ==============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product  [Admin only]
  5. Delete a product  [Admin only]
  6. Update a product price  [Admin only]
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 4

--------------------------------------------------------------------------------
  ACCESS DENIED
--------------------------------------------------------------------------------
  A Regular User may only view and search records.
  Option 4 needs the Admin role.

============== MAIN MENU  (Regular User: shopper_01) ==============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product  [Admin only]
  5. Delete a product  [Admin only]
  6. Update a product price  [Admin only]
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 5

--------------------------------------------------------------------------------
  ACCESS DENIED
--------------------------------------------------------------------------------
  A Regular User may only view and search records.
  Option 5 needs the Admin role.
```

Screen 5 — Role restrictions — continued (part 2 of 2)

Midterm — ./main · Regular User blocked from admin actions (part 2 of 2)

```
=============== MAIN MENU  (Regular User: shopper_01) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product  [Admin only]
  5. Delete a product  [Admin only]
  6. Update a product price  [Admin only]
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
=====================================================================
  Enter your choice (0-9): 2

-------------------------------------------------------------------------------
  BINARY SEARCH BY PRODUCT ID
-------------------------------------------------------------------------------
  Product ID to find: 342

  Searching 8 sorted records:
      step 1: left=0 right=7 mid=3 (ID 201)  -> 201 < 342, discard left half
      step 2: left=4 right=7 mid=5 (ID 276)  -> 276 < 342, discard left half
      step 3: left=6 right=7 mid=6 (ID 342)  -> MATCH

  FOUND after 3 comparison(s):
    Product ID:   342
    Stock Code:   84406B
    Description:  CREAM CUPID HEARTS COAT HANGER
    Quantity:     8
    Unit Price:   $2.75
    Stock Value:  $22.00

  A linear search would need up to 8 comparisons.

=============== MAIN MENU  (Regular User: shopper_01) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product  [Admin only]
  5. Delete a product  [Admin only]
  6. Update a product price  [Admin only]
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
=====================================================================
  Enter your choice (0-9): 0

-------------------------------------------------------------------------------
  EXIT
-------------------------------------------------------------------------------
  saved 8 records to "inventory.txt"
  Goodbye, shopper_01.
```

Screen 6 — Update, delete, and writing both .txt files

A price is updated through the pointer the binary search returned, a record is deleted and the gap closed, then option 7 writes both files: `inventory.txt` for the next run and `inventory_report.txt` for a person to read. The table afterwards shows the changes.

Midterm — ./main · Update, delete, and writing both .txt files (part 1 of 3)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
==============================================================
  Enter your choice (0-9): 6

--------------------------------------------------------------------------------
  UPDATE A PRODUCT PRICE
--------------------------------------------------------------------------------
  Product ID to update: 389

  Current price of ASSORTED COLOUR BIRD ORNAMENT: $1.69
  New unit price:   $1.49

  Updated through the pointer returned by the search:
    Product ID:   389
    Stock Code:   84879
    Description:  ASSORTED COLOUR BIRD ORNAMENT
    Quantity:     32
    Unit Price:   $1.49
    Stock Value:  $47.68

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
```

Screen 6 — Update, delete, and writing both .txt files — continued (part 2 of 3)

Midterm — ./main · Update, delete, and writing both .txt files (part 2 of 3)

```
  0. Exit
============================================================
  Enter your choice (0-9): 5

------------------------------------------------------------------------------
  DELETE A PRODUCT
------------------------------------------------------------------------------
  Product ID to delete: 150

  Deleted product ID 150. 7 records remain.

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 7

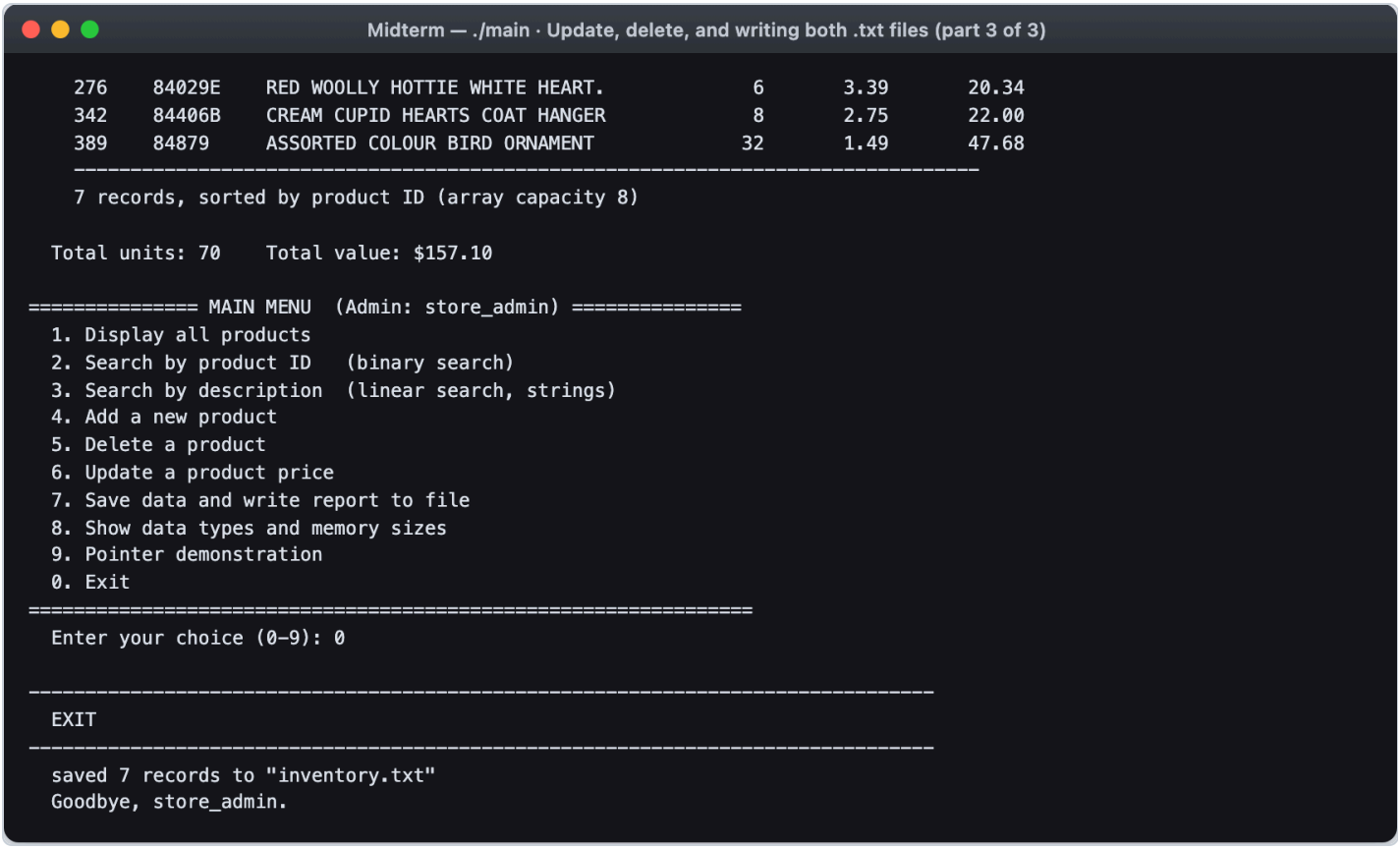
------------------------------------------------------------------------------
  WRITE BOTH .TXT FILES
------------------------------------------------------------------------------
  OK: saved 7 records to "inventory.txt"
  OK: wrote report to "inventory_report.txt"

  inventory.txt   holds the raw records for the next run.
  inventory_report.txt   holds a formatted report for a person to read.

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 1

------------------------------------------------------------------------------
  ALL PRODUCTS (array kept sorted by product ID)
------------------------------------------------------------------------------
    ------------------------------------------------------------------------
    ID     STOCK     DESCRIPTION                          QTY   UNIT ($)   VALUE ($)
    ------------------------------------------------------------------------
    105    71053     WHITE METAL LANTERN                    6       3.39       20.34
    118    84029G    KNITTED UNION FLAG HOT WATER BOTTLE    6       3.39       20.34
    201    85123A    WHITE HANGING HEART T-LIGHT HOLDER     6       2.55       15.30
    233    22633     HAND WARMER UNION JACK                 6       1.85       11.10
```

Screen 6 — Update, delete, and writing both .txt files — continued (part 3 of 3)



Screen 7 — Data types and data sizes

Every type the program uses, measured at run time with `sizeof`, in bytes and bits, with the range of `int` and the precision of `float` against `double`. This is the evidence for the choice of `double` rather than `float` for money.

Midterm — ./main · Data types and the bytes each one occupies (part 1 of 2)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

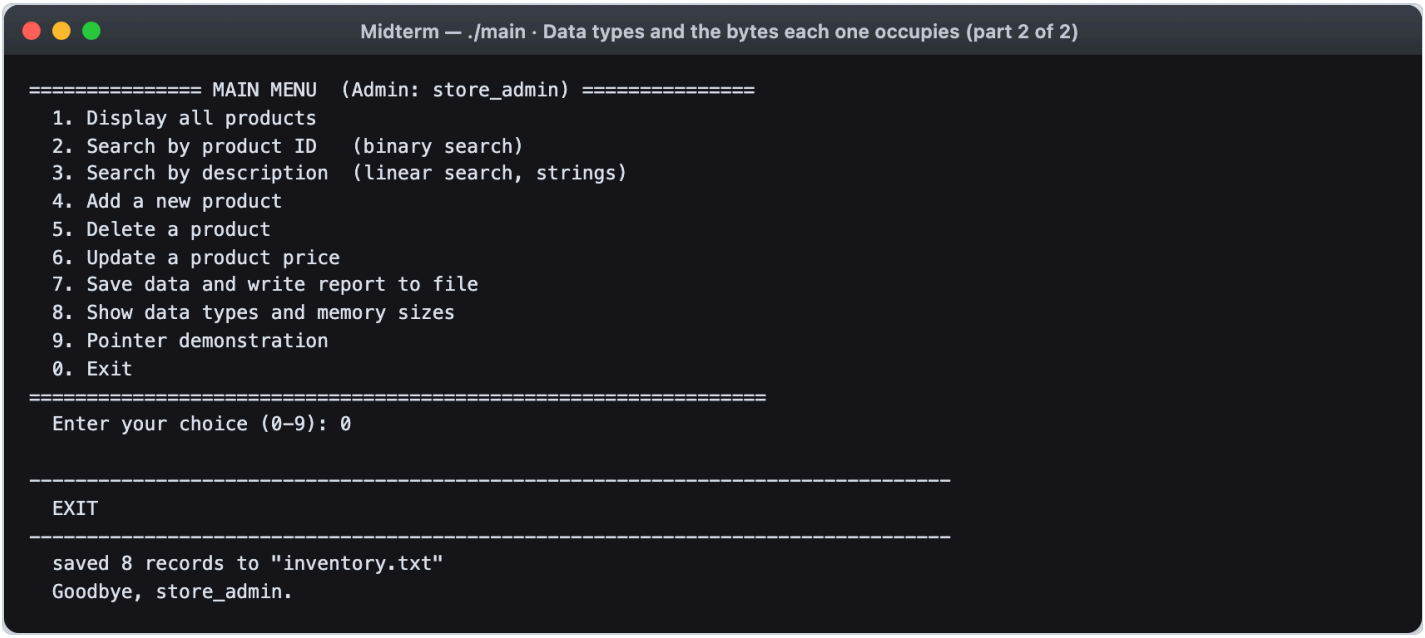
============== MAIN MENU  (Admin: store_admin) ==============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 8

--------------------------------------------------------------------------------
  DATA TYPES AND DATA SIZES (measured with sizeof)
--------------------------------------------------------------------------------
  TYPE                  BYTES   BITS    WHAT THIS PROGRAM USES IT FOR
  ------------------------------------------------------------------------
  bool                  1       8       true/false results from setters
  char                  1       8       single characters, the | separator
  short                 2       16      (not used, shown for comparison)
  int                   4       32      productId, quantity, array indexes
  long                  8       64      (not used, shown for comparison)
  long long             8       64      (not used, shown for comparison)
  float                 4       32      (not used: too few digits for money)
  double                8       64      unitPrice, stock values, totals
  std::string           24      192     stockCode, description, usernames
  Product *             8       64      pointer to the dynamic array
  Product               72      576     one inventory record
  Inventory             16      128     pointer + count + capacity

  Ranges of the whole-number and decimal types:
    int      -2147483648 to 2147483647
    double   about 15 reliable decimal digits
    float    about 6 reliable decimal digits  <- why money uses double, not float

  Note: sizeof(std::string) is the size of the object itself. The
  characters of a long string live separately on the heap, which is
  why every description costs the same 24 bytes inside a Product.
```

Screen 7 — Data types and data sizes — continued (part 2 of 2)



Screen 8 — Pointer walk across the dynamic array

A pointer starts at the first record and steps forward one record at a time. The printed addresses rise by exactly 72 bytes, which is `sizeof(Product)`. The screen then reads the same value three equivalent ways and uses pointer subtraction to recover an index.

Midterm — ./main · Pointer walk across the dynamic array (part 1 of 2)

```
================================================================================
            STORE INVENTORY MANAGEMENT SYSTEM  -  C++ Midway Project
                     CIS-25 Programming Using C++ | H Cheng
================================================================================

[Startup] Reading the data file...
  OK: loaded 8 records from "inventory.txt"

--------------------------------------------------------------------------------
  SELECT A ROLE
--------------------------------------------------------------------------------
  1. Admin        (view, search, add, delete, update)
  2. Regular User (view and search only)
  Choose a role (1-2): 1
  Enter your username: store_admin

  Signed in as store_admin with the role: Admin

=============== MAIN MENU  (Admin: store_admin) ===============
  1. Display all products
  2. Search by product ID   (binary search)
  3. Search by description  (linear search, strings)
  4. Add a new product
  5. Delete a product
  6. Update a product price
  7. Save data and write report to file
  8. Show data types and memory sizes
  9. Pointer demonstration
  0. Exit
============================================================
  Enter your choice (0-9): 9

--------------------------------------------------------------------------------
  POINTER DEMONSTRATION
--------------------------------------------------------------------------------
    The array lives on the heap. 'items' holds its first address.
    sizeof(Product) = 72 bytes, so each step
    of the pointer moves that many bytes forward.

    INDEX   ADDRESS           OFFSET    ID      DESCRIPTION
    ------------------------------------------------------------
    0       0x1015cc0f0       0         105     WHITE METAL LANTERN
    1       0x1015cc138       72        118     KNITTED UNION FLAG HOT WATER BOTTLE
    2       0x1015cc180       144       150     SET 7 BABUSHKA NESTING BOXES
    3       0x1015cc1c8       216       201     WHITE HANGING HEART T-LIGHT HOLDER
    4       0x1015cc210       288       233     HAND WARMER UNION JACK
    5       0x1015cc258       360       276     RED WOOLLY HOTTIE WHITE HEART.
    6       0x1015cc2a0       432       342     CREAM CUPID HEARTS COAT HANGER
    7       0x1015cc2e8       504       389     ASSORTED COLOUR BIRD ORNAMENT

    Three ways to read the SAME first description:
      items[0].getDescription()    -> WHITE METAL LANTERN
      (*(items + 0)).getDescription() -> WHITE METAL LANTERN
      (items + 0)->getDescription()   -> WHITE METAL LANTERN

    Reading one single value through a pointer:
      pointer to last record: 0x1015cc2e8
      last->getQuantity()     = 32 units
```

Screen 8 — Pointer walk across the dynamic array — continued (part 2 of 2)

